SPECULATIVE DECODING FOR AGENT WORKLOADS / RESEARCH & ENGINEERING MEASURED

RESULTS, WITH THE CORRECTIONS INCLUDED

How predictable is a coding agent?

A small model drafts. A larger model verifies. Speculative Decoding for Agent Workloads measures how much of an agent's work can take that shortcut—and what it takes to make the shortcut correct.

Workload Local coding-agent traffic **Main target** Qwen2.5-1.5B-Instruct **Evaluation** 14 held-out runs

71.2%

Distribution overlap for the LoRA drafter.
95% run-bootstrap interval: 70.61–71.87%.
Measured on 56,990 scored token positions.

Structure helps.

Tool calls have greater overlap than prose or thinking. Simple lookup already captures some of that repetition.

Runtime benefit remains open.

Corrected decoding results are partial. A controlled speed benchmark has not been recorded yet.

Reading this edition. An October 6 audit fixed sampling, cache work and confidence intervals. The figures here use the new held-out alpha results and the corrected judge rerun. The sweep is shown as a partial snapshot. The old report's speedup claims are withdrawn from this edition.

01 What Speculative Decoding for Agent Workloads does

Speculative Decoding for Agent Workloads is an experiment platform for speculative decoding on software-agent traffic. It connects a real workload, several draft strategies, a custom decoder and a serving stack so that an attractive prediction can be tested against actual generation.

A language model normally generates one token at a time. A token might be part of a word, punctuation or a fragment of code. Each new token depends on the previous ones, so the next model call has to wait. Speculation lets a cheaper drafter propose a short continuation and asks the target model to check the whole continuation in one call.

The motivating workload is this machine's local Qwen 27B coding agents. Their transcripts include prose, thinking, code and structured tool traffic. Those transcripts supply the data; **the experiments in this report use smaller 1.5B and 0.5B targets**. They do not measure an acceleration of the original 27B agents.

One workload, two paths through the system

The measurement path produces evidence. The serving path makes the decoder usable by an API client.

01 / COLLECT	→ 02 / DRAFT	→ 03 / VERIFY	→ 04 / MEASURE
Agent traces → corpus	**Lookup or learned model**	**JAX target + decoder**	**Alpha, sweep & judge**
Classify blocks, separate fenced code, deduplicate, tokenize and split by run.	N-grams, a distilled student, LoRA and a DPO adapter propose continuations.	Score draft tokens together. Accept a prefix, repair a rejection, roll back cache state.	Estimate distribution overlap, measure decoding steps, and check output quality.

Serving path **API client** → **Go gateway** → **FastAPI engine / optional vLLM backend** → **Streamed completion**

Implementation: `speculative_agent_workloads/data/`, `speculative_agent_workloads/drafters/`, `speculative_agent_workloads/jaxlm/`, `speculative_agent_workloads/eval/`, `speculative_agent_workloads/sweep.py`, `gateway/`.

Model runtime	A Qwen2 implementation in JAX: grouped-query attention, rotary positions, RMS normalization, a gated MLP and a fixed-capacity key/value cache. PyTorch adapters are merged into the JAX weights for decoding.
Serving & visibility	The Go gateway handles API-key authentication, per-key rate limits, model routing and a circuit breaker. It streams responses and exposes Prometheus metrics; Grafana displays latency, tokens, acceptance and backend health.
Experiment tooling ⌂ Hub	Ray schedules sweep cells. Small JSON/Parquet artifacts feed a React explorer. A separate benchmark compares configurations in one process after compilation warm-up.

Deployment tooling	Dockerfiles, a Helm chart, a host-only kind check and Terraform/GKE configuration exist. They are implementation artifacts; this snapshot does not establish a production deployment or a GPU performance result.

02 A faster path must keep the same distribution

The target stays responsible for the output. A draft token is a proposal; accepting it is a probability calculation.

At a given position, let `p` be the target's next-token distribution and `q` the distribution that actually produced the draft. For sampled decoding, the engine accepts a proposed token `x` with probability `min(1, p(x)/q(x))`. On the first rejection it samples a replacement from the normalized positive difference `max(0, p − q)`. If every draft survives, it samples a bonus token from the target.

This accept-and-repair rule preserves the target distribution under its assumptions. Greedy decoding has a simpler rule: keep the proposed token if it equals the target's top choice. At positive temperature, distribution equality is the goal; two runs need not produce the same text or consume random numbers in the same order. Leviathan et al., 2023; Chen et al., 2023.

Follow one speculative step

A synthetic example, with three drafts. The probabilities illustrate the rule; these are not corpus tokens or measured outcomes.

Committed suffix `draft 1 ✓` `draft 2 ✓` `replacement` ~~`draft 3`~~

The target cache keeps the accepted prefix; the drafter rolls back its rejected state. The replacement is committed but remains **pending in the target cache**. The next call processes it together with the next draft block.

Position	Target p(x)	Draft q(x)	Uniform u	u × q(x)	Test
Draft 1	0.40	0.30	0.60	0.18	0.18 < 0.40 → keep
Draft 2	0.35	0.40	0.50	0.20	0.20 < 0.35 → keep
Draft 3	0.12	0.30	0.70	0.21	0.21 ≥ 0.12 → repair

A rejection commits the accepted prefix plus one replacement. Later drafts are discarded because they were conditioned on the rejected token. All-accepted steps can emit k + 1 tokens, subject to the output limit.

▼ **The interesting implementation detail: a pending token**

The target cache stores every committed token except the newest one. A step feeds `[pending_token] + drafts` to the target. Logit row `j` predicts draft `j`; the final row predicts the bonus token. After accepting `A` drafts, the engine keeps the pending token and that accepted prefix in its cache. The replacement or bonus becomes the next pending token.

Cache rollback changes the logical length rather than recomputing the prefix. Values beyond that length can remain in the allocated buffers; the attention mask must keep them invisible. The drafter has its own cache and must roll back to the same committed history. This convention removed a second target forward that used to decode the bonus separately. See `1e9f5d3`.

> **Kernel scope.** Triton verify and alpha kernels exist with PyTorch references. The alpha evaluator can use the fused alpha kernel on CUDA. The current JAX decoder performs verification in its own loop; the existence of the fused verify kernel does not establish that serving uses it.

03 Data that looks like an agent's work

The original corpus groups content into thinking, text, fenced code, tool calls and tool results. Blocks are deduplicated and capped at 4,096 tokens. Entire runs belong to a single split, so adjacent blocks from the same session cannot cross the train/evaluation boundary.

The first validation and test splits each had only three runs. An audit also found that one run dominated their token mass. The newer temporal hold-out adds 14,974 blocks, 4.24 million tokens and 14 runs from later traffic, excluding runs already present and exact `(kind, text)` duplicates from the older splits.

Most training tokens are thinking or tool output

Composition by token count in the original training split.



Tool results	Thinking	Tool calls	Text	Fenced code
43.6%	32.4%	17.5%	6.1%	0.5%
6.85M tokens	5.08M tokens	2.74M tokens	0.96M tokens	0.07M tokens

Source: `results/corpus_stats.json`. Tool results are inputs read by an agent; their predictability does not directly become generation time saved.

Split	Runs	Blocks	Tokens	Role in this edition
Train	26	45,257	15,697,302	Drafter training

⌂ Hub

Split	Runs	Blocks	Tokens	Role in this edition
Val	3	667	236,985	Original validation
Test	3	566	252,500	Original test
Temporal hold-out	14	14,974	4,238,167	Current alpha / decoding / judge evaluation

What the headline actually measures

The alpha evaluator scores 700 selected blocks from the temporal hold-out, containing 56,990 token positions. It selects blocks across runs, uses blocks of at most 256 tokens and reconstructs context from preceding selected blocks within a 1,024-token budget. It then scores the recorded prefix at every position—often called **teacher forcing**.

Alpha is the overlap between the two probability distributions:

$$\alpha = \Sigma_x \min(p(x), q(x)) = 1 - TV(p, q)$$

Interpretation

An alpha of 0.71 means about 71% of the distributions' probability mass overlaps, averaged over this evaluation. With correctly sampled proposals at those contexts, it corresponds to expected acceptance. It is neither 71% next-token accuracy against the corpus nor a promise that 71% of a live draft block will survive.

Uncertainty

The saved estimates are means of 1,000 bootstrap replicates. Each replicate samples whole runs with replacement and pools their token counts. Resampling runs respects session dependence; it does not eliminate uneven run sizes. The intervals describe this sampled workload.

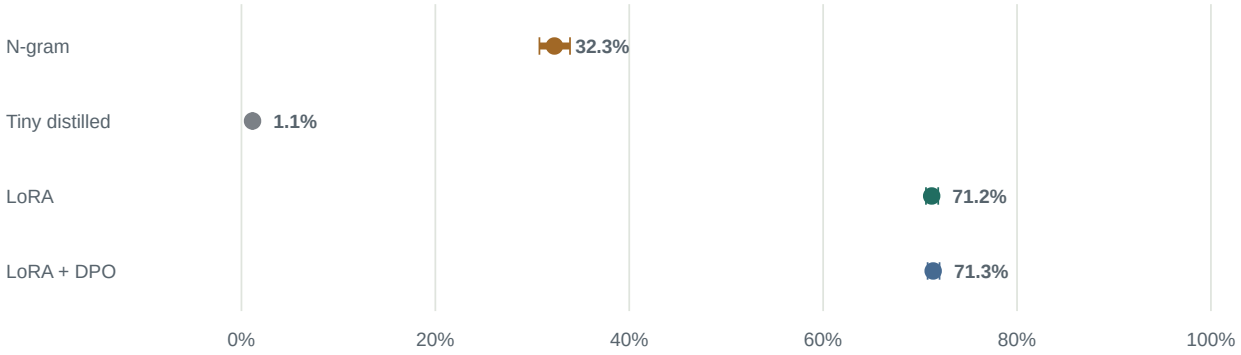
> **Scope limits.** The saved alpha distributions use unscaled softmax, corresponding to temperature 1. The corrected sweep uses 0 and 0.7, a different target and generated continuations. The selected alpha blocks contain no `code` row, so this edition makes no held-out code-specific alpha claim. No claim of generalization to other agents, models or repositories follows from these data.

04 A learned drafter captures much more overlap

On the held-out workload, LoRA reaches 71.2% overlap against the 1.5B target. N-gram lookup captures roughly a third. The tiny student captures very little.

Overall distribution overlap

Qwen2.5-1.5B-Instruct target · same 700 selected blocks · 14 runs · 95% run-bootstrap intervals



Drafter	Overlap α	95% interval	Proposal coverage
N-gram	32.28%	30.74–33.89%	73.9%
LoRA	71.20%	70.61–71.87%	100.0%
LoRA + DPO	71.35%	70.78–72.01%	100.0%
Tiny distilled	1.14%	1.09–1.20%	100.0%

Coverage is the fraction of scored positions for which a drafter proposes a token. N-gram alpha includes positions without a proposal as zero; model drafters cover almost every valid position. Source: `results/alpha/{ngram,lora,dpo,tiny}.json`.

The DPO adapter's point estimate is higher by **0.153 percentage points**. The intervals overlap substantially. Separate intervals alone cannot decide whether a paired difference is significant; the saved artifacts do not include the run-level paired difference needed to establish that claim. This edition reports a small observed difference, with the effect unresolved.

Tool traffic has more structure to exploit

Distribution overlap, with 95% intervals under each estimate.

Content kind	Runs / positions	N-gram	Tiny distilled	LoRA	LoRA + DPO
Text	14 runs 2,149 positions	19.8% 13.6–26.8%	1.9% 1.6–2.2%	62.8% 60.2–66.0%	63.1% 60.6–66.3%
Thinking	13 runs 18,273 positions	16.3% 14.6–18.2%	1.6% 1.5–1.7%	64.3% 63.5–65.2%	64.7% 63.9–65.6%
Tool results	13 runs 12,634 positions	27.3% 23.9–30.8%	1.2% 1.1–1.3%	72.0% 70.5–73.3%	72.1% 70.6–73.4%
Bash calls	13 runs 22,279 positions	47.7% 45.8–49.6%	0.7% 0.6–0.7%	76.4% 75.6–77.3%	76.4% 75.6–77.4%
Edit calls	4 runs 1,348 positions	56.9% 53.9–61.4%	0.6% 0.6–0.7%	81.7% 77.1–83.2%	81.6% 77.1–83.1%
Read calls	5 runs 307 positions	56.0% 45.0–61.0%	0.5% 0.3–0.7%	77.4% 66.2–83.0%	77.4% 66.5–83.0%

Darker shading means a higher estimate. All model pairs use the same support. Read and Edit calls have only five and four supporting runs, respectively; their wider intervals deserve attention. On a phone, choose a drafter to keep the estimates and intervals readable.

The pattern is consistent with repetitive tool syntax and task-specific structure helping prediction. This is an interpretation of the observed differences. Establishing that training specifically learned agent structure would also require an unadapted 0.5B baseline and a matched comparison on non-agent text; those comparisons are not established by the current held-out alpha artifacts.

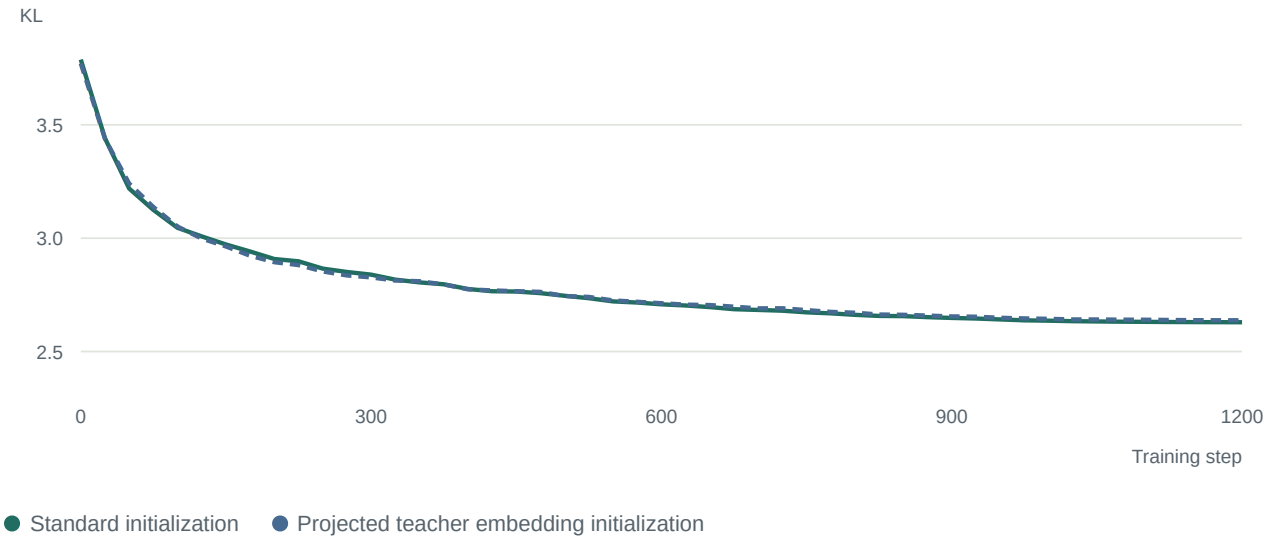
05 Four strategies, different tradeoffs

Strategy	How it proposes	Why it is interesting
N-gram lookup	Matches recent sequences of one to four tokens against earlier committed context; proposes the following token.	No neural model forward. Repetition can be cheap to exploit. No self-match with unseen future text.
Tiny distilled	A 4-layer, 256-wide JAX student; shared token vocabulary, its own trainable embeddings.	43.8M parameters. Distillation stores the teacher’s top-64 probabilities plus aggregate tail mass rather than every logit.

Strategy	How it proposes	Why it is interesting
LoRA	A Qwen2.5-0.5B base model adapted to agent traffic, rank 16, scaling 32, attention and MLP projections.	Small trainable matrices change the base weights. The code provides FSDP training and a GPU-only QLoRA path.
LoRA + DPO	Preference training on target-corrected tokens versus rejected draft tokens; TRL DPO with β = 0.1.	Attempts to train toward acceptance. The pair generator has an open validity issue described below.

The student learned the training objective

Validation KL over 1,200 training steps. Lower is better; KL is a distribution-distance objective.



Standard run: 3.788 → 2.629, a 30.6% decrease. The teacher-initialized run ends at 2.638. These are historical training curves, not held-out acceptance measurements. Sources: `results/distill_curve.json`, `results/distill_curve_tinit.json`.

Despite lower KL, the standard student reaches only **1.14% held-out overlap**. A compressed training objective can improve while the downstream metric stays poor. The student's small capacity, limited training and block-only training context are plausible contributors; this experiment does not isolate their effects.

For LoRA, the deployment bridge is concrete: `W ← W + (scaling/rank) · B · A`. The sweep reads the saved adapter tensors and adds those low-rank updates to the attention and MLP matrices in the JAX model. Training and inference can therefore use different frameworks without a separate PyTorch drafter in the decoding loop.

Open code finding: DPO pair construction needs an audit

The current pair builder chooses `d = argmax(q)`, then applies an acceptance test using the soft `q[d]` and residual `p − q`. That does not describe sampling from `q`. It also prefills a sequence and then feeds its last token again when starting cached generation. These observations come from `speculative_agent_workloads/drafters/dpo.py`, not a rerun of training. The main decoder's October 6 fix does not change this separate code path. Rebuild and check pairs before treating the observed DPO difference as evidence for a correct acceptance-aware training method.

---

06 Overlap is potential. Decoding is the test.

The corrected sweep generates 32 tokens after short held-out prompts, with draft lengths $k \in \{1, 2, 4, 8\}$ and temperatures 0 and 0.7. It uses a Qwen2.5-0.5B-Instruct target. Here the learned drafter and target have similar parameter counts, so this grid is useful for decoder behavior but is a poor basis for a large speedup claim.

Rows are recorded per prompt. A finished prompt is appended to a sidecar file; the driver folds those records into Parquet on completion or restart. The report reads both forms, deduplicates by configuration and prompt index, and leaves the experiment files untouched. Partial work is visible without rerunning a cell.

Corrected decoding sweep: completed and partial cells

0.5B-Instruct target · 32 generated tokens per prompt · 115 prompts in a complete cell

Drafter	k = 1	k = 2	k = 4	k = 8
N-gram	1.18 115 prompts	1.25 115 prompts	1.30 115 prompts	1.33 115 prompts
Tiny distilled	1.08 115 prompts	1.09 115 prompts	1.10 115 prompts	1.10 115 prompts
Identity control	2.00 115 prompts	2.91 115 prompts	4.57 115 prompts	8.00 115 prompts
LoRA	1.60 115 prompts	Partial 94/115 prompts	Partial 51/115 prompts	Partial 1/115 prompts
LoRA + DPO	Partial 43/115 prompts	Pending 0 prompts	Pending 0 prompts	Pending 0 prompts

26 complete 8 partial 8 pending 42 configurations, including two target-only baselines

Tokens per step = total emitted / total target steps. Completed cells use the same prompt set. Partial estimates use different subsets and cannot be compared as finished results. Hatched cells mark partial work; an empty cell is not a zero.

Sources: `results/sweep.parquet` and its 8 saved sidecars. Target-only baselines are pending in this snapshot. The 1.5B sweep and controlled speed benchmark are also pending; no measured speedup is reported here.

Generation moves off the recorded corpus as soon as a continuation diverges. An average teacher-forced alpha cannot describe every subsequent context. The sweep also counts every scored draft in its denominator, including draft positions after the first rejection that cannot be committed. A change in `k` can therefore change the observed accepted/drafted fraction without changing the first-token overlap.

The identity control uses the target itself as the drafter. Its acceptance should be one. With `k = 8`, the ideal full step emits nine tokens, but a 32-token output cap requires four steps: `32 / 4 = 8` tokens per step. Finite output limits matter when comparing an analytic expectation with a short generation.

Explore the tradeoff between acceptance and draft cost

An illustrative cost model. These controls do not load measured speed data.

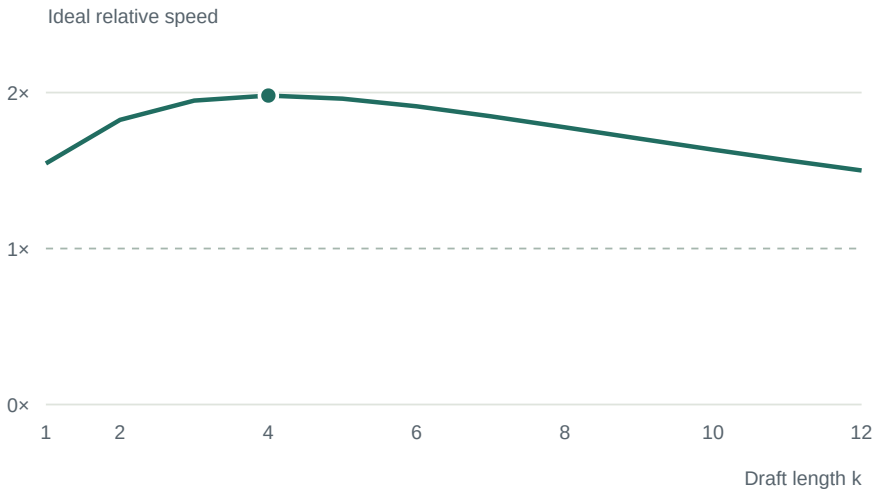
Acceptance α	0.70
Draft / target cost c	0.10
Draft length k	4

1.98×

Ideal relative speed at the selected k

2.77 tokens / target call

Best k in this model: 4



$$E[\text{tokens / call}] = 1 + \alpha + \alpha^2 + \ldots + \alpha^k$$
$$\text{Ideal relative speed} = E[\text{tokens / call}] / (1 + k \cdot c)$$

This simplification assumes constant, independent acceptance, a proposal at every position and a block verification cost equal to a single target decode. It excludes prefill, output caps, EOS, compilation, cache overhead and contention. Leviathan et al., analysis of generated tokens and wall time.

> **The next measurement.** `tools/bench_speedup.py` warms up model shapes, runs configurations in repeated interleaved passes and compares each with target-only decoding on the same prompts. Its result file, `results/speedup_bench.json`, is absent. A drafter's acceptance or a timing ratio from shared Ray workers is insufficient evidence of an end-to-end speedup.

07 Quality checks support the algorithm, with limits

The current test suite includes greedy equality, cache parity with a fresh prefill, sampled-distribution checks and a count of target forwards per step. The added positive-temperature tests exercise an identity drafter and the second-token marginal of a mismatched drafter; they check paths that a first-token or greedy-only test can miss.

A separate judge rerun uses held-out prompts on the fixed engine: 0.5B-Instruct target, DPO drafter, `k = 4`, temperature 0.7, 16 generated tokens. The local `qwen-agent` judge receives blind comparisons in both presentation orders, with cached responses and a 150-call cap. These are short-output preference checks, not software-task completion tests.

The corrected judge rerun

Wins, ties and losses from the speculative output's perspective.

Speculative vs target only



34 / 2 / 43 wins / ties / losses · 79 verdicts over 44 prompts
Win rate 43.0% · reported Wilson 95% interval 32.7%–54.0%
Both orders valid: 39 prompts; disagree: 8. File's reported order disagreement: 18.2%.

Speculative vs drafter alone



19 / 0 / 10 wins / ties / losses · 29 verdicts over 25 prompts
Win rate 65.5% · reported Wilson 95% interval 47.3%–80.1%
Both orders valid: 12 prompts; disagree: 1. File's reported order disagreement: 4.0%.

Spec wins Tie Spec loses

Source: `results/judge.json`, generated October 6. The file records 150 calls used and 18 invalid responses. Comparison totals above count included verdicts. The file's reported order-disagreement rate divides by all paired entries, including invalid orders; the counts show pairs with both orders valid.

The speculative-versus-target win-rate interval includes 50%. This does not prove equivalence or non-inferiority. Both orders judge the same prompt, so verdicts are correlated; the file's Wilson intervals treat verdicts as independent and should be read as diagnostic summaries. The order-disagreement count also shows that judging is sensitive to presentation.

Distribution preservation rests on the acceptance rule and implementation checks. A judge can help detect an obvious quality change, but it cannot replace those checks or validate long coding tasks from 16-token continuations.

⌂ Hub

08 What the fixes taught us

Several of the most useful results came from learning which measurements could be trusted. The following changes are grounded in the repository's commit history.

SEP 30
7a84891 ↗

A one-token shift can look like a bad model

Failure. The evaluator scored token `i` with logit row `i`, which predicts token `i + 1`. The resulting low overlap was a measurement error.

Fix. Align logits to next-token labels; mark the first position invalid when no preceding context exists. Known-answer checks compare cross-entropy with Hugging Face's internally shifted loss and test repeated versus shuffled text.

Lesson: give a metric a case with a known answer before interpreting an unexpected number.

SEP 30
ae371e3 ↗

Correct tokens can hide incorrect cache state

Failure. The model drafter decoded the last committed token twice; on full acceptance, its position marker also failed to advance correctly.

Fix. Align proposal positions and rollback. Regression tests compare the drafter's key/value cache with a fresh prefill and check its next proposal. October 6 subsequently replaced the layout with the pending-token convention.

Lesson: check the state after both rejection and full acceptance, not just the text returned by one step.

OCT 06
1e9f5d3 ↗

A plausible q is not necessarily the proposal distribution

Failure. Drafts were chosen greedily but reported with soft probabilities, without matching temperature. The accept-and-repair rule then used a distribution that had not produced the token. Positive-temperature output was biased toward greedy choices.

Fix. Sample model drafts from `softmax(logits / T)` and report that exact distribution; at zero temperature, use a one-hot greedy distribution. The decoder configures the drafter's temperature and seed. The same change removes the extra target forward.

Lesson: stochastic correctness includes the proposal mechanism, temperature, random keys and state transitions. An identity drafter at positive temperature is a valuable control.

OCT 06

0158d68 ↗

A transposed bootstrap made uncertainty disappear

Failure. The multinomial call created `n_runs` replicates with `n_boot` draws each. Each replicate spread many draws almost evenly across the same few runs, so intervals collapsed around the estimate.

Fix. Create `n_boot` replicates, each drawing `n_runs` sessions. A two-run regression fixture with very different acceptance rates requires an interval spanning both.

Lesson: validate the resampling unit and array shape. Small error bars do not establish a small effect when the uncertainty calculation is wrong.

OCT 06

5f98414 ↗

The workload and comparison need to match the claim

Failure. The initial splits contained too few independent sessions; first-N prompt selection could concentrate a kind in one session. Concurrent sweep wall times were also used for speedup claims.

Fix. Add a later hold-out, pick prompts round-robin across runs, lengthen generation from 16 to 32 tokens, add a 1.5B target option and build an interleaved benchmark with a target-only baseline.

Lesson: acceptance, runtime savings and generalization require different controls. A larger sample of nearly identical context does not supply those controls.

OCT 06

48cf15b ↗

Completion semantics belong in the correctness story

Failure. Serving kept generating after end-of-sequence tokens and always returned `finish_reason: stop`, even at the length limit.

Fix. Stop at the tokenizer's EOS markers, exclude them from completion text, report `stop` versus `length`, and expose a no-drafter model for comparisons.

Lesson: verify the protocol that callers observe, alongside the decoder's numerical behavior.

09 What would close the remaining questions?

Question	Evidence needed	Snapshot status
Does speculation save wall time?	Controlled head-to-head benchmark on the fixed engine, matched prompts and target-only decoding.	Benchmark file absent.
Does a smaller drafter help a larger target?	1.5B decoding sweep with the 0.5B adapters, followed by controlled timing.	1.5B sweep absent.
Does DPO improve acceptance?	Audit and rebuild preference pairs; compute a paired run-level difference, with an unadapted baseline.	Small point difference; effect unresolved.

Question	Evidence needed	Snapshot status
Do corpus scores predict generation?	Score the fixed engine’s generated trajectories at the same positions and temperature.	Stored trajectory artifact predates the audit.
Does the GPU path improve latency?	Actual GPU kernel and serving benchmarks, with target-only and speculative arms.	Host script exists; no GPU results.
Can a new developer reproduce it?	Clean-clone checks with explicit skips for absent data and models, then the full verification gate.	Open task records four missing-data test failures.

> Speculative Decoding for Agent Workloads establishes useful overlap on this held-out agent workload and implements the machinery to test speculation. The next decisive result is a controlled runtime comparison on the corrected engine.

---

10 Sources, snapshots & reproduction

This report is a frozen snapshot of revision `e874bd1` and the result files present at generation time. Aggregate downloads include the full revision, source hashes, recorded confidence intervals and sweep counts. They contain no prompts, transcript text, token sequences or run identifiers. The standalone HTML works offline and needs no chart library or external font.

Snapshot generated 2026-10-06 · 19:05:19 UTC. New experiment results do not change this page automatically.

Rebuild this report

```
# From the Speculative Decoding for Agent Workloads project, with its Python dependencies available:
python tools/build_presentation.py

# Outputs:
# report/index.html   — portable report
# report/data.json    — aggregate evidence, metadata and commit diffs
# report/metrics.csv  — full-precision metrics, intervals and status

# Optional copy into the local loop reports server:
python tools/build_presentation.py --publish ../../reports/speculative-agent-workloads-report.html
```

The builder reads stored artifacts; it does not train, generate or judge. It rejects unexpected alpha support and refuses to silently keep the pending-benchmark narrative if those result files become available.

⌂ Hub

Commit evidence

Commit	Date	Change
`7a84891`	2026-09-30	alpha: fix off-by-one logit alignment, known-answer tests, 1.5B default target
`ae371e3`	2026-09-30	spec: fix ModelDrafter propose double-decode and full-accept rollback; regression tests
`1e9f5d3`	2026-10-06	spec: lossless T>0 model drafting, one target forward per step
`0158d68`	2026-10-06	alpha: fix run bootstrap (CIs were ~100x too narrow)
`5f98414`	2026-10-06	eval: temporal hold-out split, head-to-head speed bench, 1.5B sweep
`48cf15b`	2026-10-06	serve: stop at EOS, real finish_reason, '+none' drafter

Research references

Leviathan, Kalman & Matias (ICML 2023), Fast Inference from Transformers via Speculative Decoding — sampling rule and analytic cost model.
Chen et al. (2023), Accelerating Large Language Model Decoding with Speculative Sampling — distribution-preserving speculative sampling.